2 Cyber Range Description

This chapter describes the syntax of the cyber range description file used as a CyRIS input, which is written using the YAML format [4]. For parsing the description file we use the PyYAML library for the Python programming language [5]. The description is divided into three sections, as explained below:

- `host_settings` contains information about the hosts the cyber range is to be deployed on, including an id, a management address, and an account.
- `guest_settings` provides information about the base images used to instantiate the cyber range, and defines all the content of the cyber range that needs to be prepared by CyRIS.
- `clone_settings` gives details about the cloning phase, and contains an unique range id, along with other information, such as the actual hosts that the cyber range is to be deployed on, the number of guests, and their network topology.

Although in the next sections we shall provide several examples of how to use each of the cyber range description parameters, we also recommend checking the files in the `examples/` directory of the distribution, in particular `full.yml`, for practical use cases.

2.1 `host_settings` section

The first part of the cyber range description file is `host_settings`, which includes information about the hosts that will be involved in the cyber range deployment. The first host specified in this section is designated as the master host, which processes the description file, and creates the cyber range across itself and the other hosts. The parameters users need to specify for each host, are presented next, together with their roles, expected data types and values.

`id` The id of the host, used internally to refer to it.

- Data type: string
- Value: any value

mgmt_addr The management address of the host, used to communicate with it.

- Data type: string
- Value: any value

virbr_addr The address of the KVM virtual bridge on the host[1] (this parameter is not needed when using AWS virtualization).

- Data type: string (IPv4 address representation)
- Value: any value (default = `192.168.122.1`)

account The account on the host that will be used by CyRIS.

- Data type: string
- Value: any value

The number of hosts specified in this section is not limited. Below is an example of a description consisting of two hosts, in which the first of them, with id `host_1`, becomes the master host.

```
- host_settings:
  - id: host_1
    mgmt_addr: 172.16.24.1
    virbr_addr: 192.168.122.1
    account: cyuser
  - id: host_2
    mgmt_addr: 172.16.24.2
    virbr_addr: 192.168.122.1
    account: cyuser
```

2.2 `guest_settings` section

The second and main part of the description file is `guest_settings`, which provides information about the base images and content installation. Each guest section starts with global information about the guest and the base image used to create it (such as guest id, base image file, etc.). This is followed by a set of operations that need to be performed on it in order to create the desired guest content. This section explains first about the parameters for specifying global information regarding a guest, then goes into detail about how to use CyRIS functions to perform various operations for setting it up.

[1]The KVM virtual bridge typically appears on the host as an interface with the name `virbr0` and the default IP address `192.168.122.1`.

2.2.1 Global information

The necessary global information parameters for a guest, their roles, expected data types and values are as follows.

id The id of the guest, which by convention reflects its role in the cyber range (desktop, web server, database, and so on).

- Data type: string
- Value: any value

ip_addr An optional parameter for specifying the base image IP address used during the configuration process of this guest. If not provided, CyRIS automatically assigns a default IP address for the base image[2].

- Data type: string (IPv4 address representation)
- Value: 192.168.122.N (with N between 2 and 254)

basevm_host Specifies the id of the host on which the base image is located. In the current implementation, CyRIS only supports base images that are located on the master host.

- Data type: string
- Value: master host id

basevm_config_file When using KVM virtualization, this parameter specifies the location of the XML configuration file used to define the KVM base image (this is not needed when using AWS virtualization).

- Data type: string (an absolute file path)
- Value: any value

basevm_type Specifies the type of the base image; CyRIS currently supports creating virtual machines locally via KVM virtualization technology or remotely via the AWS on-demand cloud computing platform.

- Data type: string
- Value: { `kvm` | `aws` }

basevm_os_type Specifies the OS type of the base image (see also Section 2.5.1).

- Data type: string
- Value: One of the choices below, depending on the value of `basevm_type`:

[2]Currently the convention is as follows: for the nth base image in the `guest_settings` section (starting from 0), the default IP address is generated as `192.168.122.`X, where $X = 100 + n$.

- In case the base VM type is "`kvm`": { `windows.7` }; if `basevm_os_type` is completely omitted, generic Linux is assumed
- In case the base VM type is "`aws`": { `amazon_linux` | `amazon_linux2` | `red_hat` | `ubuntu_16` | `ubuntu_18` | `ubuntu_20` }

`tasks` Specifies a list of operations that will be performed on the base image. Details about these tasks will be given in the next sections.

The number of guests in this section is also not limited. Below is an example of a description consisting of two guests called `desktop` and `webserver`. Note that the `desktop` guest is specified first, and the parameter `ip_addr` is not included, hence its IP address will be assigned the value `192.168.122.100`. Similarly, the `webserver` will be assigned the IP address `192.168.122.101`.

```
- guest_settings:
  - id: desktop
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_desktop.xml
    basevm_type: kvm
    tasks:
      ...
  - id: webserver
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_webserver.xml
    basevm_type: kvm
    tasks:
      ...
```

CyRIS also provides a number of functions, which help instructors to perform various operations on a base image, so as to create their desired security-related content. These functions are divided into five main categories, which are *system configuration*, *tool installation*, *incident emulation*, *content management*, *clone management* (cf. Table 1.1).

The first four categories are for creating independent content on each guest, and are specified within the subsection `tasks` of the section `guest_settings` in the description file. The last function group is about cloning and setting up network service, thus it is specified in the section `clone_settings` (see Section 2.3).

The remainder of this section describes the setup operations that the instructors can ask CyRIS to perform on the guest base images within the `tasks` subsection.

2.2.2 `add_account` task

This function is used for creating a new account on the base image. The necessary parameters, their roles, expected data types and values are as follows.

`account` Specifies the user name of the account to be created.

- Data type: string
- Value: any value

passwd Specifies the password of the account to be created.

- Data type: string
- Value: any value

full_name Optional parameter for specifying the full name of the user.

- Data type: string
- Value: any value

The number of new accounts to be created is not limited. Below is an example of a description for creating a new account on the guest "desktop" (the global information is omitted). The new account has the name "daniel", the password is "danielpass", and the user's full name is "Daniel Radcliffe".

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - add_account:
      - account: daniel
        passwd: danielpass
        full_name: Daniel Radcliffe
```

2.2.3 modify_account task

This function is for editing information of existing accounts on the base image. The necessary parameters, their roles, expected data types and values are as follows.

account Specifies the user name of the account being modified.

- Data type: string
- Value: name of an existing account

new_account Specifies a new name for the account when the intention is to change the account name.

- Data type: string
- Value: any value

new_passwd Specifies a new password for the account when the intention is to change the account password.

- Data type: string
- Value: any value

Depending on the need to modify only the account name, only the password, or both, this operation needs only two or all the three parameters above. Below is an example of a description for modifying the password of the "root" account to "abcd1234" on the guest "desktop" (the global information is omitted).

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - modify_account:
      - account: root
        new_passwd: abcd1234
```

2.2.4 `install_package` task

This function is for installing packages on the base image. The necessary parameters, their roles, expected data types and values are as follows.

`package_manager` Optional parameter for specifying a package management tool to be used to install the desired package. If this parameters is not provided, `yum` will be used as package manager.

- Data type: string
- Value: { `yum` | `apt-get` | `chocolatey` }

`name` Specifies the name of the package to be installed.

- Data type: string
- Value: any value

`version` Optional parameter for specifying the package version. By default, CyRIS will install the latest version of the package available in the repository.

- Data type: string
- Value: any value

Below is an example description for installing the package called "wireshark" with version "1.8.10" on the guest "desktop". Note that since the default value for the package manager is "yum", the corresponding line could have been omitted.

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - install_package:
      - package_manager: yum
        name: wireshark
        version: 1.8.10
```

2.2.5 emulate_attack task

This function is for emulating an attack on the base image. Currently CyRIS supports three kinds of attacks, namely SSH dictionary, DoS and DDoS attacks, and this functionality is available on the CentOS 7 operating system only. The first type of attack is done via stand-alone emulation, since logs and traces are consequently generated automatically in the system. For the latter two attacks, one needs to add other tasks for capturing the packet traces, as it will be discussed in Section 2.2.6.

This section describes the necessary parameters, their roles, expected data types and values for an SSH dictionary attack emulation.

attack_type Specifies the attack type. As mentioned above, SSH dictionary attack is the only option at this moment.

- Data type: string
- Value: `ssh_attack`

target_account Specifies the account on the base image that the attack should target.

- Data type: string
- Value: name of an existing account

attempt_number Specifies how many attempts of the attack should be performed. Note that the bigger the attempt number, the longer the emulation takes.

- Data type: int
- Value: any value

attack_time Optional parameter for specifying the date when the attack is supposed to have taken place. If this information is provided, CyRIS modifies the system date accordingly before starting the attack emulation. Otherwise, the current date will be used.

- Data type: string
- Value: $YYYY$-MM-DD (or $YYYYMMDD$)

Below is an example of a description for an SSH dictionary attack which targets the account named "daniel" on the base image "desktop". The attack includes 54 attempts, and is considered to have taken place on "November 18th, 2017".

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - emulate_attack:
      - attack_type: ssh_attack
        target_account: daniel
        attempt_number: 54
        attack_time: 2017-11-18
```

2.2.6 `emulate_traffic_capture_file` task

This function is for emulating network sniffing activities, and capturing traffic in PCAP format, e.g., to include attack traces. In the current stage, CyRIS supports generating traffic traces for SSH dictionary, DoS and DDoS attacks. Below are the necessary parameters, their roles, expected data types and values.

`format` Specifies the format of the traffic capture file. Currently only the PCAP format is supported.

- Data type: string
- Value: `pcap`

`file_name` Specifies the absolute path of the file the traffic capture should be written to.

- Data type: string
- Value: any value

`attack_type` Specifies the type of the attack from among the supported ones.

- Data type: string
- Value: { `ssh_attack` | `dos_attack` | `ddos_attack` }

`attack_source` Specifies the IP address from where the attack is supposed to have been initiated. This parameter is only necessary for the `ssh_attack`.

- Data type: string (IPv4 address representation)
- Value: any value

`noise_level` Specifies how much normal traffic "noise" needs to be added into the capture file, for the purpose of making it more challenging to detect the attack pattern.

- Data type: string
- Value: { `low` | `medium` | `high` }

Below is an example of a description for emulating an SSH dictionary attack and a DDoS attack. The traces are written to the files "traffic1.pcap" and "traffic2.pcap", respectively, and the noise levels are "medium" in both cases. The SSH attack is supposed to originate from the IP address "69.89.31.226".

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - emulate_traffic_capture_file:
      - format: pcap
        file_name: /home/trainee/traffic1.pcap
        attack_type: ssh_attack
        attack_source: 69.89.31.226
```

```
        noise_level: medium
      - format: pcap
        file_name: /home/trainee/traffic2.pcap
        attack_type: ddos_attack
        noise_level: medium
```

2.2.7 emulate_malware task

The malware emulation function is used to run a dummy malware process, which has no actual malicious behavior, hence it is safe even for beginner training. The corresponding command is placed in the file `/etc/rc.d/rc.local`, so that it is run automatically as root when the OS boots.

The malware emulation functionality currently implemented in CyRIS has two distinct execution modes:

1. Perform a calculation to increase the CPU load, similar to the symptoms of some of the malicious processes.

2. Listen to a specified port, so as to reproduce the behavior of a malware that opened a backdoor.

Below are the necessary parameters for configuring the malware emulation mode, their roles, expected data types and values.

name Specifies the name the dummy malware should use.

- Data type: string
- Value: any value

mode Specifies the mode for the dummy malware from among the supported ones.

- Data type: string
- Value: { `dummy_calculation` | `port_listening` }

cpu_utilization Specifies the percentage of CPU time that the malware will consume when using the `dummy_calculation` mode.

- Data type: integer
- Value: [0, 100]

port Specifies the number of the port the malware will listen to in case the malware is configured to use the `port_listening` mode.

- Data type: integer
- Value: [1, 65535]

Below is an example of a description for emulating two malware processes. The first one, called "daemon" uses the "dummy_calculation" mode, and will cause a 40% CPU utilization. The second one, called "spyeye", uses the "port_listening" mode, and will listen on port "1052".

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - emulate_malware:
      - name: daemon
        mode: dummy_calculation
        cpu_utilization: 40
      - name: spyeye
        mode: port_listening
        port: 1052
```

2.2.8 `copy_content` task

The copy content function is for copying data, both files and directories, from an external location to the base images, for instance in order to create a desired configuration, place files for trainees to investigate, etc. Below are the necessary parameters, their roles, expected data types and values.

`src` Specifies the absolute path *on the master host* of the file/directory to be copied.

- Data type: string
- Value: any

`dst` Specifies the absolute path *on the base image* of the location where the data is to be copied to.

- Data type: string
- Value: any

Below is an example of a for copying the directory "penetration_testing" and the text file "flag.txt" from the master host to the base image. The destinations are the directories "/bin/cyberrange" and "/home/trainee", respectively.

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - src: /home/cyuser/database/penetration_testing
      dst: /bin/cyberrange
    - src: /home/cyuser/database/flag.txt
      dst: /home/trainee
```

2.2.9 execute_program task

The execute program function is for executing scripts/programs, such as Python, bash, or Ruby scripts, on the base image. The programs must already exist before executing this task, for instance by a copy operation as discussed above. Below are the necessary parameters, their roles, expected data types and values.

program Specifies the absolute path of the program location.

- Data type: string
- Value: any

args Optional parameter for specifying any arguments that are needed to run the above program. Arguments should be separated by spaces.

- Data type: string
- Value: any

interpreter Specifies an interpreter for running the program from among the supported ones.

- Data type: string
- Value: { `python` | `bash` | `ruby` }

execute_time Optional parameter indicating whether the program should be executed *after* the cloning phase, otherwise it is not necessary. This is useful in case the execution depends on the properties of an actual virtual machine instance in the final cyber range, such as its IP address, etc.

- Data type: string
- Value: `after_clone`

Below is an example of a description for executing two bash scripts. The first one, "install_tool.sh", is executed *before* the cloning phase to install a certain tool in the guest base image; it also requires two arguments, "/tool/path" and "parameter". The second script, named "prepare.sh", will be executed *after* the cloning phase, for instance to do some machine-dependent configuration.

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - execute_program:
      - program: /bin/cyberrange/penetration_testing/install_tool.sh
        args: /tool/path parameter
        interpreter: bash
      - program: /bin/cyberrange/penetration_testing/prepare.sh
        interpreter: bash
        execute_time: after_clone
```

2.2.10 `firewall_rules` task

The firewall rule operation is for specifying firewall rules; CyRIS will automatically set up the firewall rules specified in the provided file(s). Only the `iptables` firewall program is currently supported. Below are the necessary parameters, their roles, expected data types and values.

`rule` Specifies the absolute path of the file containing the list of firewall rules to be configured.

- Data type: string
- Value: any

Below is an example in which two files are used to define firewall rules, one for forwarding actions, and the other one for I/O operations. The files are named "firewall_ruleset_forwarding" and "firewall_ruleset_input_output", respectively.

```
- guest_settings:
  - id: desktop
    ... <global information> ...
    tasks:
    - firewall_rules:
      - rule: /home/cyuser/database/firewall_ruleset_forwarding
      - rule: /home/cyuser/database/firewall_ruleset_input_output
```

Firewall rule files should use the format of the `iptables-save` command. An example of such a file is shown below, which includes rules to open the ports 20 and 21 needed to allow active-mode FTP connections to a server.

```
# Allow active-mode FTP connections
-A INPUT -p tcp --dport 21 -j ACCEPT
-A OUTPUT -p tcp --sport 20 -j ACCEPT
```

2.3 `clone_settings` section

The third main section in the cyber range description file is `clone_settings`, which gives details about the cloning phase. Each cyber range has a unique id, along with other information, such as the hosts that cyber range is deployed on, the guests which would be included in the cyber range, and the network topology.

2.3.1 Parameters

The necessary parameters that users need to specify in this section are presented next, together with their roles, expected data types and values.

range_id Specifies a unique id of the cyber range, which is used to differentiate between cyber ranges created on the same host. Currently, this id is used as the first byte in the network address of the cyber range, therefore one must avoid values that cannot be assigned to the first byte in an IP address.

- Data type: integer
- Value: [1, 254]

hosts Specifies the hosts, described as a YAML list, that the cyber range is to be deployed on. The following parameters should be provided for each host.

host_id The id of the host the cyber range should be deployed on. This host id must be already defined in the section `host_settings` of the description file.

- Data type: string
- Value: an existing host id

instance_number The number of cyber range instances of this type that are to be deployed on the host. Note that cyber range performance can decrease if too many instances are deployed on a given host.

- Data type: integer
- Value: any positive value

guests A list of guests that are to be cloned from the descriptions provided in the `guest_settings`. The following parameters should be indicated for each guest.

guest_id The id of the guest that the virtual machine is to be cloned from.

- Data type: string
- Value: an existing guest id

number The number of virtual machines of this type that are to be cloned.

- Data type: integer
- Value: any positive value

forwarding_rules An optional list of forwarding rules for communication among network segments. This parameter is only specified when a guest plays the role of forwarding traffic in the network (such as a firewall machine, gateway machine, etc.). Each rule must contains four parameters: `src` and `dst` for the source and destination segment, and `sport` and `dport` for the source and destination port of the traffic.

- Data type: string
- Value: `src=`SRC `dst=`DST `sport=`$SPORT$ `dport=`$DPORT$

entry_point An optional keyword indicating whether the current guest is to be used as entry point for accessing the cyber range[3]. One and only one occurrence of this keyword is necessary.

[3]For each cyber range instance, CyRIS will set up an SSH tunnel, so that trainees can connect from outside the cyber range to the designated entry point guest.

- Data type: string
- Value: `yes`

topology Specifies the network topology among the virtual machines in the cyber range given as a list of networks. The following parameters should be specified for each network.

type The type of the network in the cyber range. At the current stage, only a "custom" type is supported, but "ring" type or "dumbbell" type specifications can also be imagined.

- Date type: string
- Value: `custom_type`

networks A list of network segments in this cyber range network, including the name of the segment, a list of guest members, and a gateway, as follows.

name The name of the network segment, such as "office", "internal", "external", etc.

- Data type: string
- Value: any value

members A comma-separated enumeration of members in this network segment. For each member one should specify the guest id and its network interface, connected by a period character, for example, "desktop.eth0", where "desktop" is a guest id, and "eth0" is an interface name.

- Data type: string
- Value: `guest_id.interface`, `guest_id.interface`, ...

gateway Optional parameter to indicate the default gateway for members in this segment. If this parameter is given, then the IP address of the indicated guest/interface will be set as the default gateway for all the members in the segment. Otherwise, the default gateway address is set automatically by appending `1` to the three first bytes of the segment network address, e.g., `<byte1>.<byte2>.<byte3>.1`.

- Data type: string
- Value: `guest_id.interface` (default is explained above)

2.3.2 Address assignment

The following should be taken into account regarding the `clone_settings` section of the cyber range description file:

- As mentioned above, the `range_id`, is used not only to differentiate among cyber ranges, but also as the first byte in the network address of the cyber range. Therefore, it is important to assign it an appropriate value from a network address representation point of view.

- Due to the complex network topology a cyber range might have, and also to avoid human mistakes, the IP addresses for individual virtual machines in the cyber range are assigned automatically by CyRIS as follows:
  - The network address for the entire cyber range: `<range_id>.1.1.1/8`
  - The network address for a cyber range instance: `<range_id>.<instance_id>.1.1/16` (with the value of `instance_id` starting from 1)
  - The network address of each network segment, depending on the order in which the segments are defined in the description file: `<range_id>.<instance_id>.<segment_no>.1/24` (with the value of `segment_no` starting from 1)
  - The IP address of each cloned virtual machine in a segment, depending on the order in which it appears in the `members` enumeration of the `networks` subsection of `topology` in `clone_settings`: `<range_id>.<instance_id>.<segment_no>.<member_no>` (the value of `member_no` starts from 2, and it also depends on how many guests of a certain type are specified in the `number` field of the `guests` subsection of `hosts` in `clone_settings`)
  - The default gateway IP address of every member in a network segment (if the `gateway` parameter in the `networks` subsection of the section `topology` in `clone_settings` is not specified): `<range_id>.<instance_id>.<segment_no>.1`

Below is an example of a description for a cyber range with id "112" that is to be deployed on host "host_1". A number of 2 instances of this cyber range will be deployed, each of them consisting of 2 "desktop" machines and 1 "firewall" one, which is also designated as the entry point to the instance. The desktops are all connected via their interface "eth0" to the same network segment, named "office", and they will use the "eth0" interface IP address of the "firewall" machine as the default gateway IP address.

```
- host_settings:
  - id: host_1
    ...<host info>...

- guest_settings:
  - id: desktop
    ...<guest info>...
  - id: firewall
    ...<guest info>...

- clone_settings:
  - range_id: 112
    hosts:
    - host_id: host_1
      instance_number: 2
      guests:
```

```
    - guest_id: desktop
      number: 2
    - guest_id: firewall
      number: 1
      entry_point: yes
    topology:
    - type: custom
      networks:
      - name: office
        members: desktop.eth0
        gateway: firewall.eth0
```

Considering the IP address assignment rules that we have introduced above, the IP address of each machine in the cyber range will be configured as follows:

- **Cyber range 112, instance #1**:
  - **desktop #1**:
    - IP address: `112.1.1.2`
    - Default gateway: `112.1.1.4`
  - **desktop #2**:
    - IP address: `112.1.1.3`
    - Default gateway: `112.1.1.4`
  - **firewall**:
    - IP address: `112.1.1.4`
    - Default gateway: `112.1.1.1`
- **Cyber range 112, instance #2**:
  - **desktop #1**:
    - IP address: `112.2.1.2`
    - Default gateway: `112.2.1.4`
  - **desktop #2**:
    - IP address: `112.2.1.3`
    - Default gateway: `112.2.1.4`
  - **firewall**:
    - IP address: `112.2.1.4`
    - Default gateway: `112.2.1.1`

2.4 Detailed example

Here we provide an example of a complete description for creating a realistic cyber range. The target setup has a DMZ network topology as shown in Figure 2.1, and the actual CyRIS description for this cyber range in included below the figure.

The main characteristics of the DMZ cyber range, as indicated in the main sections of the description, are as follows:

- `host_settings`: The cyber range is to be deployed on a single physical host, which is given the id "host_1", followed by management and virtual bridge IP addresses and account information;
- `guest_settings`: A total of five guests are defined: "firewall", "dnsmail", "filesrv", "dbsrv", and "desktop". For each guest the base image properties are provided, followed by a series of setup tasks, such as adding accounts, installing packages, copying files, and executing scripts;
- `clone_settings`: A single cyber range instance will be created on the host, which will contain one guest of each type. For the "firewall" guest some forwarding rules are defined, and it is designated as the entry point for connecting to the cyber range instance. The network topology is composed of 3 segments: one for external servers, one for internal servers, and one for office, each with the corresponding guests attached to them.

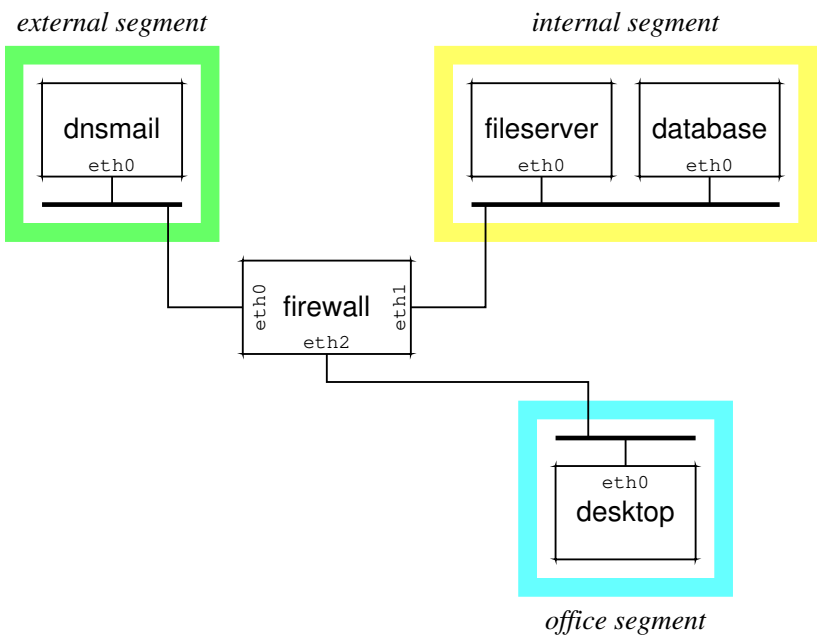

Figure 2.1: Target DMZ network topology for the example description.

```
---
- host_settings:
```

```
  - id: host_1
    mgmt_addr: 172.16.1.7
    virbr_addr: 192.168.122.1
    account: cyuser

- guest_settings:
  - id: firewall
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_firewall.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: robot.abc
        passwd: abcrb1357
    - modify_account:
      - account: root
        new_passwd: abcd.1234

  - id: dnsmail
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_dnsmail.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: robot.abc
        passwd: abcrb1357
    - modify_account:
      - account: root
        new_passwd: abcd.1234
    - install_package:
      - package_manager: yum
        name: wget
      - package_manager: yum
        name: telnet

  - id: filesrv
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_filesrv.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: robot.abc
        passwd: abcrb1357
    - modify_account:
      - account: root
        new_passwd: abcd.1234
    - install_package:
      - package_manager: yum
        name: samba samba-client samba-common
      - package_manager: yum
        name: wget

  - id: dbsrv
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_dbsrv.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: robot.abc
        passwd: abcrb1357
    - modify_account:
      - account: root
```

```
        new_passwd: abcd.1234
    - install_package:
      - package_manager: yum
        name: wget

  - id: desktop
    basevm_host: host_1
    basevm_config_file: /home/cyuser/images/basevm_desktop.xml
    basevm_type: kvm
    tasks:
    - add_account:
      - account: daniel
        passwd: JamesBond
    - install_package:
      - package_manager: yum
        name: nmap
      - package_manager: yum
        name: telnet

- clone_settings:
  - range_id: 124
    hosts:
    - host_id: host_1
      instance_number: 1
      guests:
      - guest_id: firewall
        number: 1
        forwarding_rules:
        - rule: src=office,external dst=internal.dbsrv dport=3306
        - rule: src=office,external dst=internal.filesrv dport=139,445
        - rule: src=office dst=external dport=25,53
        entry_point: yes
      - guest_id: dnsmail
        number: 1
      - guest_id: filesrv
        number: 1
      - guest_id: dbsrv
        number: 1
      - guest_id: desktop
        number: 1
      topology:
      - type: custom
        networks:
        - name: external
          members: dnsmail.eth0
          gateway: firewall.eth0
        - name: internal
          members: filesrv.eth0, dbsrv.eth0
          gateway: firewall.eth1
        - name: office
          members: desktop.eth0
          gateway: firewall.eth2
```

2.5 Additional information

CyRIS is under active development, and its functionality is continuously extended. This section provides more details about the support provided by the current version.